March, 2026

PRJ_5AI00_TP

Blockchain - Synod Project

Final Report

Authors: Daniele Famà, Marcin Porwisz

---

1 Problem Statement

The goal of this project is to study the consensus problem in an asynchronous distributed system. Consensus is a fundamental problem in distributed computing in which multiple processes must agree on a common value, even in the presence of failures and without relying on a global clock.

In this project, we implement and study an *obstruction-free consensus* (OFC) algorithm.

1.1 Consensus Operation

The algorithm provides a single operation called `propose(v)`, where the input value v belongs to the set $V = \{0, 1\}$.

When a process invokes `propose(v)`, we say that the process *proposes* the value v. The operation may return either:

- a value $v' \in V$, in which case the process *decides* v', or
- a special value *abort*, which indicates that the current attempt did not lead to a decision.

If the operation returns *abort*, the process may invoke `propose` again and attempt to reach a decision in a later execution of the algorithm. In general, multiple processes may invoke `propose` concurrently.

1.2 Correctness Properties

The algorithm must satisfy the following properties.

Validity Every decided value must have been proposed by some process.

Agreement No two processes decide different values.

Obstruction-Free Termination

- If a correct process invokes `propose`, the invocation eventually returns either a decision or the value *abort*. In the latter case, the process may try again by invoking `propose` another time.
- If a correct process reaches a decision, other correct processes cannot keep aborting forever. Eventually they must also stop aborting and reach a decision.
- If at some point only one correct process continues invoking `propose` while the others stop interfering, then that process will eventually reach a decision.

Intuitively, obstruction-freedom guarantees that progress is made when a process runs without interference from other processes. Interference refers to the situation where several processes execute the algorithm concurrently and exchange messages, which may prevent a process from immediately reaching a decision.

1.3 System Model

The algorithm is executed in the following distributed environment.

- The system consists of N asynchronous processes. Each process has a unique identifier known to all other processes. Asynchronous execution means that processes may run at arbitrary speeds and there is no global clock that synchronizes their actions.
- Processes communicate through reliable asynchronous point-to-point channels. This means that messages are sent directly from one process to another and may be delivered after an arbitrary delay, but they are not lost and are eventually delivered.
- Up to $f < N/2$ processes may experience crash failures. A crash failure means that a process permanently stops executing its algorithm and no longer sends or receives messages.
- A process that never crashes during an execution is called a *correct* process.

1.4 Evaluation Metrics

In a consensus protocol, consensus is reached when all correct processes eventually decide the same value. This follows from the agreement and termination properties of the algorithm.

In our experiments, we focus on measuring the time until the first decision occurs in the system, defined as the latency between the first invocation of `propose` and the moment when the first process reaches a decision.

This metric serves as a practical proxy for consensus latency. Once a value is decided, the agreement property ensures that no other process can decide a different value, and the decision will eventually propagate to all correct processes.

2 High-Level Description of the Implementation

To solve the consensus problem described in the previous section, we implement the *Synod* algorithm. Algorithm 1 presents the pseudocode of the protocol executed by each process.

Synod is a classical consensus algorithm that forms the core of the Paxos family of protocols. It allows a set of distributed processes to agree on a value despite asynchronous execution and

Algorithm 1 Synod: code for process p_i

```
Local variables:
    ballot := i − n; proposal := nil; readballot := 0;
    imposeballot := i − n; estimate := nil; states := [nil, 0]^n

Upon propose(v) // process p_i proposes v:
    proposal := v; ballot := ballot + n; states := [nil, 0]^n
    send [READ, ballot] to all

Upon received [READ, ballot'] from p_j:
    if readballot > ballot' or imposeballot > ballot' then
        send [ABORT, ballot'] to p_j // reject the ballot number
    else
        readballot := ballot'
        send [GATHER, ballot', imposeballot, estimate] to p_j // accept the ballot number

Upon received [ABORT, ballot] from some process:
    return abort

Upon received [GATHER, ballot, estballot, est] from p_j:
    states[p_j] := [estballot, est]

Upon |states| > n/2: // received a majority of responses
    if ∃states[p_k] = [estballot, est] with estballot > 0 then
        select states[p_k] = [est, estballot] with highest estballot
        proposal := est // choose a potentially decided value
    states := [nil, 0]^n
    send [IMPOSE, ballot, proposal] to all

Upon received [IMPOSE, ballot', v] from p_j :
    if readballot > ballot' or imposeballot > ballot' then
        send [ABORT, ballot'] to p_j
    else
        estimate := v; imposeballot := ballot'
        send [ACK, ballot'] to p_j

Upon received [ACK, ballot] from a majority:
    send [DECIDE, proposal] to all

Upon receive [DECIDE, v]:
    send [DECIDE, v] to all
    return v
```

Figure 1: Synod algorithm

possible crash failures. Each process in the system repeatedly invokes the `propose(v)` operation. The algorithm proceeds through a sequence of communication phases in which processes exchange messages and gradually converge toward a common value.

The main idea behind the Synod algorithm is that processes try to convince a majority of the system to accept a value. A process proposing a value first asks the other processes about previously accepted values. Based on the responses it receives, it selects a value that is safe to propose and then asks the other processes to accept that value.

If a majority of processes acknowledge the proposal, the value becomes decided and the decision is broadcast to the entire system. Once a value has been decided, all correct processes eventually learn and adopt that decision.

2.1 Overview of the Algorithm

The Synod algorithm organizes the execution into numbered *ballots*. A ballot represents one attempt to reach consensus. Ballot numbers are totally ordered and increase over time, allowing processes to distinguish newer attempts from older ones. If an attempt fails, a process may start a new ballot with a higher number.

Each ballot proceeds in two main phases.

- **Read phase.** A process initiating a ballot sends a `READ` message to all other processes to collect information about previously accepted values.
- **Impose phase.** After receiving responses from a majority of processes, the proposer selects a value (either its own proposal or a previously accepted one) and sends an `IMPOSE` message to ask the other processes to accept that value.

If a majority of processes acknowledge the imposed value, the proposer broadcasts a `DECIDE` message indicating that the value has been chosen. Because multiple processes may initiate ballots concurrently, conflicts may occur. If a process receives a message associated with an outdated ballot number, it rejects the request by sending an `ABORT` message. In that case, the proposing process aborts its current attempt and may start a new ballot later. When a process receives a `DECIDE` message, it adopts the decided value and forwards the decision to all other processes. This ensures that once a value has been decided, the information quickly propagates through the system.

2.2 Implementation Approach

We implement the *Synod* algorithm in Java using the AKKA actor framework. The actor model is a natural choice for this project because it represents distributed processes as independent entities communicating through asynchronous message passing. The source code is available online[1].

General Structure Each process in the distributed system is implemented as an AKKA actor. A process actor maintains its own local state and communicates with the other processes by sending protocol messages. In addition to the process actors, the implementation contains a dedicated `ResultCollector` actor, whose role is to gather decision events and compute summary statistics for each experimental run.

At the beginning of an execution, each process receives the list of references to all processes (including itself). The system then launches all processes, and each one starts by choosing an initial proposal value in $\{0, 1\}$.

[1] github.com/danielefam/synod-blockchain

Mapping Synod to Actors The behaviour of a process closely follows the Synod pseudocode. Each message type appearing in the algorithm is represented by a Java message class, and each *upon receiving* clause from the pseudocode is implemented as a corresponding message handler in the actor.

A process starts a proposal attempt by invoking an internal method that corresponds to one execution of `propose(v)` (line 4 in the pseudocode). At the start of each attempt, the process increases its ballot number and broadcasts a `READ` message to all processes (lines 5–6). Ballot numbers are generated so that each process produces a unique increasing sequence, which makes it possible to distinguish newer attempts from older ones.

When a process receives a `READ` message, it either rejects the request with `ABORT` if it has already seen a higher ballot, or responds with a `GATHER` message containing its current estimate and the ballot in which that estimate was imposed (lines 7–12).

After collecting responses from a majority of processes (line 17), the proposer examines the received estimates. If at least one response contains a value previously imposed in an earlier ballot (i.e., with a positive ballot number), the proposer selects the value associated with the highest such ballot (lines 18–20). Otherwise, it keeps its own proposal. In both cases, the proposer then broadcasts an `IMPOSE` message with the selected value to all processes (line 22). The responses received from the majority do not need to agree; conflicts are resolved by selecting the value with the highest associated ballot.

When a process receives an `IMPOSE` message (line 23), it again checks whether the ballot is still valid. Since the system is asynchronous, newer ballots may have appeared since the read phase, and outdated proposals must be rejected. If the ballot is outdated, the request is rejected with `ABORT` (lines 24–25). Otherwise, the process adopts the proposed value as its current estimate and replies with `ACK` (lines 26–28).

Once the proposer receives acknowledgements from a majority of processes, it broadcasts a `DECIDE` message (lines 29–30). Upon receiving `DECIDE`, a process marks itself as decided and forwards the decision to all other processes, which ensures that the decision quickly propagates through the system (lines 31–33).

Repeated Attempts and Abort Handling The project specification allows a process to invoke `propose` multiple times. In the implementation, this is handled automatically. If a proposer receives an `ABORT` message for its current ballot, it starts a new proposal attempt with a higher ballot number, unless it has already decided, crashed, or been stopped by the leader-emulation mechanism. This models the repeated proposal behaviour required by obstruction-free consensus.

To model failures, the implementation allows a subset of processes to enter a fault-prone mode. A fault-prone process does not crash immediately; instead, whenever it handles an event, it crashes with probability α. A crashed process stops participating in the protocol and ignores further messages.

Leader Emulation The implementation also includes an emulation of leader election. After a timeout t_{le}, one process that is not fault-prone is selected to continue proposing, while all other processes receive a `HOLD` message. A held process stops initiating new proposal rounds, but it still responds to protocol messages from other processes. This mechanism creates a period in which a single process can continue without interference, thereby approximating the obstruction-free condition under which progress is guaranteed.

The timeout is implemented in the main experimental driver using AKKA's scheduler. After t_{le} milliseconds, the scheduler triggers a one-time action that selects one process that is not fault-prone and sends a `HOLD` message to every other process.

Measurement and Result Collection The implementation records the time at which each process is launched and the time at which it first receives a `DECIDE` message. Upon receiving this message, the process is considered decided and reports a `DecisionEvent` to the `ResultCollector`.

The collector aggregates these events for a single experimental run and computes the number of decisions, the first observed decision latency, the average decision latency, and whether all deciding processes agreed on the same value.

These per-run results are written to CSV files and later aggregated across repeated runs of the same configuration.

Each experiment is repeated 5 times. After launching the processes, the main method waits for a fixed grace period to allow the protocol to run, and then asks the `ResultCollector` to flush the results of the current execution. Since executions are observed for a finite time, not all processes are required to decide; therefore, a run is considered only if at least one decision is observed.

3 Performance evaluation

Performance evaluation of our implementation is based on measuring the consensus latency (the time taken for the first process to decide) and the overall liveness of the system (the number of processes that successfully reach a decision). There are multiple factors that influence these metrics, specifically the total number of processes in the system (N), the leader election timeout (t_{le}), and the probability of processes crashing (α).

3.1 Experimental setup

Our test suite is implemented such that the system parameters can be configured before starting the execution. The number of processes N is varied between 3, 10, 30, 70, and 100. The number of fault-prone processes f is defined as the maximum allowed minority, $f = \lfloor (N-1)/2 \rfloor$. To simulate varying network stability, the process crash probability α is tested at 0.0 (no crashes), 0.1 (randomized crashes), and 1.0 (immediate crashes). Finally, to analyze the impact of the leader election mechanism on enabling obstruction-free progress, the timeout t_{le} is varied across $1, 10, 100, 200$, and 1000 milliseconds. Each unique configuration is repeated 5 times. The system uses a fixed `GRACE_MS` period, which acts as a simulation window allowing the elected leader sufficient time to complete its unobstructed proposal phase before results are collected and the actor system is terminated.

3.2 Discussion

Figure 2 shows the impact of the number of nodes on the baseline latency. For smaller network sizes ($N \leq 30$), the average consensus latency remains low (between 0ms and 15ms). Because the Akka actor system delivers messages in-memory, communication is sufficiently fast for processes to reach a majority agreement before the leader election timeout (t_{le}) is triggered. As a result, consensus is often achieved without requiring the obstruction-free phase.

At larger scales ($N = 70$ and $N = 100$), we can clearly see the effect of too many processes trying to propose at the same time. As demonstrated in Figure 3, the system often gets stuck in a "livelock" where processes keep aborting each other. It is important to note that the Synod algorithm theoretically guarantees eventual consensus once a leader is unobstructed; however, our simulation is constrained by a strict execution timer (t_{le} plus a grace period). For example, with $N = 100$ and no crashes ($\alpha = 0.0$), if we set the timeout too high ($t_{le} = 1000$ms), the high volume of concurrent `READ` and `IMPOSE` messages causes the processes to perpetually reject one another

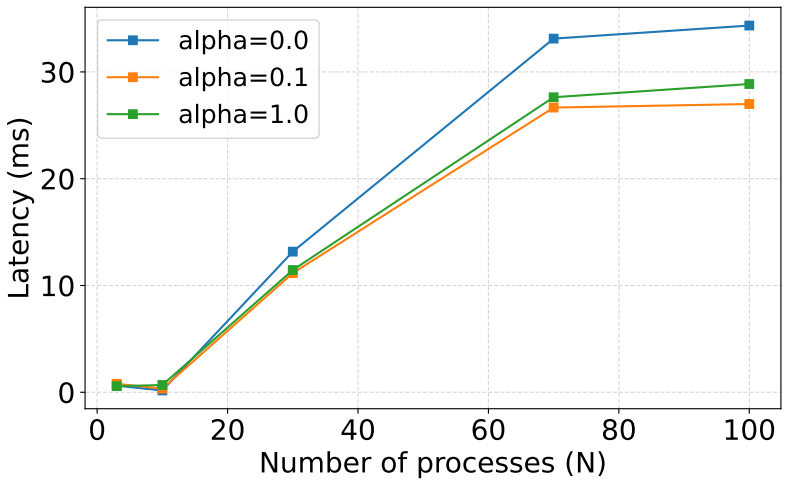


Figure 2: Average consensus latency as a function of the number of processes (N) with a fixed leader election timeout $t_{le} = 10$ms.

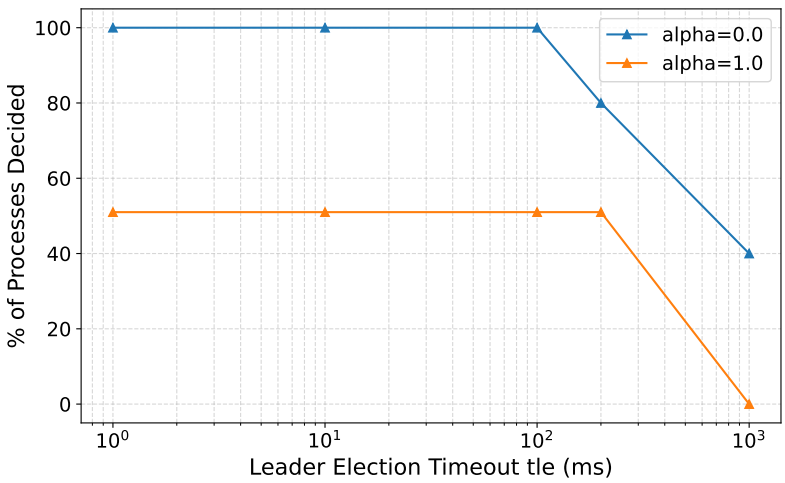
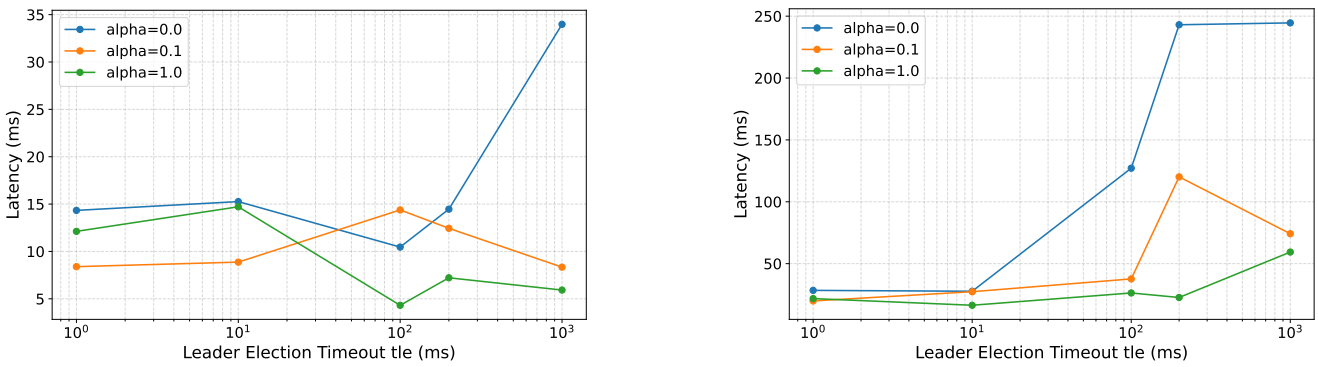


Figure 3: System liveness (percentage of successful decisions) as a function of the leader election timeout for $N = 100$.

with `ABORT` messages. This extreme contention severely floods the local Akka actor mailboxes. By the time the `HOLD` message is finally issued at $t_{le} = 1000$ms, the system is processing such a massive amount of pending messages that the newly elected leader cannot finish its unobstructed propose phase before the grace period expires. Consequently, the actor system is terminated before any decision can be reached, leading to a significant drop in the average number of decisions.

Figure 4 illustrates the latency trends for $N = 30$ and $N = 70$, showing generally *lower* (better) latency results as the crash probability α *increases*. Interestingly, the results show that increasing the crash probability α can reduce latency. For example, for $N = 100$ and $t_{le} = 10$ms, configurations with no crashes ($\alpha = 0.0$) yield an average latency of 34ms, while configurations with $\alpha = 1.0$ achieve a lower latency of 28ms. While node failures are generally detrimental to distributed systems, in an Obstruction-Free consensus environment dead nodes effectively act as silenced nodes. By immediately removing fault-prone proposers from the network, the overall message traffic and contention are drastically reduced, allowing the surviving correct processes to reach a majority more quickly.



(a) $N = 30$ (b) $N = 70$

Figure 4: Total latency as a function of leader election time (t_{le}) for fixed system sizes.

4 Conclusion

In this project, we implemented the Synod obstruction-free consensus algorithm using the Akka actor framework.

Beyond demonstrating correctness, the experimental evaluation provided practical insight into the behaviour of the protocol under varying levels of contention and failure. In particular, we observed that an improperly chosen leader election timeout (e.g., $t_{le} = 1000$ms) can significantly delay progress in large systems ($N = 100$), leading to runs in which no decision is reached within the observation window. This behaviour is caused by high contention, where multiple concurrent proposers repeatedly interfere with each other. In contrast, a smaller timeout (e.g., $t_{le} = 10$ms) effectively reduces contention by allowing one process to proceed without interference, enabling the system to reach a decision quickly.

We also observed a counter-intuitive effect of process failures on performance. In configurations with a high crash probability ($\alpha = 1.0$), the average consensus latency was lower than in failure-free runs. Although failures reduce the number of participating processes, they also decrease contention by eliminating competing proposers. As a result, the remaining correct processes can reach agreement more efficiently. This illustrates the importance of considering contention, rather than only failures, when evaluating the performance of obstruction-free algorithms.

Overall, this project highlights the delicate balance between contention, liveness, and fault tolerance in distributed consensus protocols, and demonstrates how implementation details and system parameters can significantly influence observed behaviour.